BrightLearn
DATA ANALYTICS

EXERCISE 01 · SQL FOUNDATIONS

# SQL Fundamentals: SELECT & Filtering

Your first hands-on, pen-and-paper SQL exercise. Read the table, write the query that returns the requested data, and draw the expected output table.

| COURSE | FORMAT |
| --- | --- |
| BrightLearn Basic SQL | Handwritten · Pen & Paper |
| **QUESTIONS** | **FOCUS** |
| 10 queries on 1 table | SELECT · WHERE · ORDER BY |



**Read · Query · Draw**
Start your SQL journey here.

# 00 Exercise Instructions

Welcome to your first SQL exercise. This is where every analyst, data scientist, and database developer started — learning how to read data out of a table using `SELECT` and how to narrow it down using `WHERE`.

## ★ LEARNING OBJECTIVES

By the end of this exercise you will be able to:

- Retrieve data with **SELECT** — both all columns (`SELECT *`) and specific columns.
- Remove duplicate values from results with **DISTINCT**.
- Sort results using **ORDER BY** (ascending and descending).
- Limit the number of rows returned with **LIMIT**.
- Filter rows with **WHERE**, combining conditions using **AND, OR, NOT,** and **IN**.
- Predict the exact output of a query before running it.

## FOR EACH QUESTION, DO THIS:

**1**

### WRITE THE QUERY

Hand-write the full SQL statement that produces the requested result. Use proper SQL keywords in CAPS.

**2**

### DRAW THE OUTPUT

Draw the expected output table by hand — column headers on top, rows underneath.

**3**

### CHECK COLUMNS

Make sure your output uses the exact column names listed under *Expected Columns*.

## SUBMISSION RULES

- **Format:** Handwritten on lined paper, using pen (no pencil, no typing).
- **Layout:** Number every answer to match the question (1 through 10).
- **Queries:** Write each SQL statement neatly. SQL keywords in `UPPERCASE`, table and column names in lowercase.
- **Output tables:** Draw clear column headers, then list each row beneath. Use a ruler if your handwriting needs help staying straight.
- **Originality:** All work must be your own — this exercise is graded on your understanding, not a perfect answer.

# 01 Selecting & Sorting

Start with the basics: pulling data out of the table, removing duplicates, sorting it, and limiting how many rows you get back. The `employees` table below is the data source for every question in this exercise.

▸ TABLE employees

| id | first_name | last_name | department | salary | hire_date | city |
|---|---|---|---|---|---|---|
| 1 | Alice | Green | IT | 70000 | 2020-01-10 | Johannesburg |
| 2 | Brian | Lee | HR | 45000 | 2019-03-22 | Cape Town |
| 3 | Cathy | Zulu | Finance | 65000 | 2018-07-18 | Durban |
| 4 | David | Mokoena | Marketing | 50000 | 2021-11-05 | Pretoria |
| 5 | Eva | Naidoo | IT | 72000 | 2017-09-30 | Johannesburg |

**01** **Retrieve all columns from the employees table.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department` `salary` `hire_date` `city`

**02** **Find all unique departments.**

EXPECTED COLUMNS: `department`

**03** **Retrieve first and last names ordered by salary descending.**

EXPECTED COLUMNS: `first_name` `last_name`

**04** **Retrieve the top 3 highest-paid employees.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `salary`

**★ REMINDER — SELECT ORDER MATTERS**

SQL clauses must appear in this order: `SELECT` → `FROM` → `WHERE` → `ORDER BY` → `LIMIT`. To find the "top 3 highest-paid," combine `ORDER BY salary DESC` with `LIMIT 3`.

# 02 Filtering with WHERE

Now narrow down your results. The `WHERE` clause lets you keep only the rows that match a condition — combine conditions with **AND, OR, NOT,** and **IN** for more powerful queries.

↩ **Refer to the `employees` table on the previous page** — it's the same dataset for every question in this exercise.

**05** **Find employees in the IT department.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department`

**06** **Find employees in Finance with salary > 60000.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department` `salary`

**07** **Find employees in HR or Marketing.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department`

**08** **Find employees not in IT.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department`

**09** **Find employees in IT, HR, or Finance using IN.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department`

**10** **Find employees in IT with salary > 65000 and city Johannesburg.**

EXPECTED COLUMNS: `id` `first_name` `last_name` `department` `salary` `city`

## ★ BEFORE YOU SUBMIT — CHECK YOUR WORK

- Did you write all **10** SQL queries?
- Does each output use the exact column names listed under *Expected columns*?
- Did you draw a clear output table for every query — even single-row results?
- Did you wrap text values in single quotes? E.g. `WHERE department = 'IT'`.
- Are your queries readable — keywords in CAPS, one clause per line where helpful?

# Every analyst started with SELECT *.

You're now writing the same queries that data professionals use every day. Take your time, draw your tables clearly, and predict the output before running anything. This is how SQL becomes second nature.

★ BRIGHTLEARN DATA ANALYTICS ★